CORE–3: PROGRAMMING USING C++ (UNIT – 2)

Defining Member Functions: A member function of a class is a function that has its definition or its prototype within the class definition like any other variable. It operates on any object of the class of which it is a member, and has access to all the members of a class for that object.

A member functions can be defined in two ways

- within the class definition
- Outside the class definition using **scope resolution operator (::)**

Defining a member function within the class definition declares the function **inline**, even if we do not use the inline specifier. So the **Volume()** function can be defined in two ways as follows −

Member function defined **inside** the class

```
class Box {
   public:
      double length;      // Length of a box
      double breadth;     // Breadth of a box
      double height;      // Height of a box
      double getVolume(void) {
         return length * breadth * height;
      }
};
```

Member function defined **outside** the class

```
class Box {
   public:
      double length;      // Length of a box
      double breadth;     // Breadth of a box
      double height;      // Height of a box
      double getVolume(void);
};
double Box::getVolume(void) {
   return length * breadth * height;
}
```

A member function will be called using a dot operator (**.**) on an object where it will manipulate data related to that object only.

Example to calculate the volume of a box-

```
#include<iostream>
using namespace std;
class Box {
   public:
      double length;     // Length of a box
      double breadth;  // Breadth of a box
      double height;    // Height of a box

      // Member functions declaration
      void setLength( double len );
      void setBreadth( double bre );
      void setHeight( double hei );
      double getVolume(void);
};
// Member functions definitions
void Box::setLength( double len ) {
   length = len;
}
void Box::setBreadth( double bre ) {
   breadth = bre;
}
void Box::setHeight( double hei ) {
   height = hei;
}
double Box::getVolume(void) {
   return length * breadth * height;
}

// Main function for the program
int main() {
   Box Box1; // Declaration or Create an Object
   double volume = 0.0;     // Store the volume of a box

   // box 1 specification
   Box1.setLength(6.0);   //Call member function for Box1
   Box1.setBreadth(7.0); //Call member function for Box1
   Box1.setHeight(5.0);   //Call member function for Box1

   // volume of Box 1
   volume = Box1.getVolume();  //Call member function
   cout << "Volume of Box1 : " << volume <<endl;

   return 0;
}
```

Output:

```
Volume of Box1 : 210
```

Making an outside Function Inline:

One of the objectives of OOP is to separate the details of implementation from the class definition. It is therefore good practice to define the member functions outside the class. We can define a member function outside the class definition and still make it inline by just using the qualifier inline in the header line of the function definition.

Example:

```
class item{
    .....
    public:
    void getdata(int a,float b);
};
inline void item :: getdata(int
a,float b){
    number=a;
    cost=b;
}
```

Nested Member Functions: A member function can call another member function of the same class directly without using the dot operator. This is called as *nesting of member functions*. For example,

```
#include<iostream>
using namespace std;
class nest{
        int a;
        int square_num( ){   return a* a;   }
    public:
        void input_num( ){
            cout<<"\nEnter a number ";      cin>>a;
        }
        int cube_num( ){       return a* a*a;      }
        void disp_num();
};
void nest:: disp_num(){
    int sq=square_num();   //nesting of member function
    int cu=cube_num();      //nesting of member function
    cout<<"\nThe square of "<<a<<" is  " <<sq;
    cout<<"\nThe cube of "<<a<<" is  " <<cu;
}
```

```
int main()
{
    nest n1;
    n1.input_num();
    n1.disp_num();
    return 0;
}
```

Output:

```
Enter a number 5
The square of 5 is 25
The cube of 5 is 125
```

In the above program disp_num() function calls two other member function square_ num() and cube_num(). Both are defined in different visibility mode.

Private Member Functions: Access modifiers are used to implement an important feature of Object-Oriented Programming known as **Data Hiding.** There are 3 types of access modifiers available in C++ such as **Public, Private and Protected**.

- A **private** member variable or function cannot be accessed, or even viewed from outside the class, only the class and friend functions can access private members.
- A **public** member is accessible from anywhere outside the class but within a program.
- A **protected** member variable or function is very similar to a private member but it provided one additional benefit that they can be accessed in child classes which are called derived classes.
- If we do not specify any access modifiers for the members inside the class then by default the access modifier for the members will be Private.

Example:

```
#include<iostream>

using namespace std;
class Circle {
   private:
      double radius;     // private data member

   public:
      double  compute_area() {
         return 3.14*radius*radius;
      }
};

int main() {
   Circle obj;     // creating object of the class

   // trying to access private data member
   // directly outside the class
   obj.radius = 1.5;
   cout << "Area is:" << obj.compute_area();
   return 0;
}
```

Output:

Compile time error because we are not allowed to access the private data members of a class directly outside the class.

```
#include <iostream.h>
class Circle {
   private:
      double radius;      // private data member

  public:
   void setRadious(double r){
         radious = r;
   }
   void compute_area() {
         double area = 3.14*radius*radius;
         cout << "Radius is: " << radius << endl;
         cout << "Area is: " << area;
    }
};
int main() {
   Circle obj;     // creating object of the class
   obj. setRadious(1.5);
   obj.compute_area();
   return 0;
}
```

Output:

```
Radius is: 1.5
Area is: 7.065
```

Arrays within a Class: In C++, we can have arrays as data member of a class. The member functions of the class can perform any operation on this array. For example,

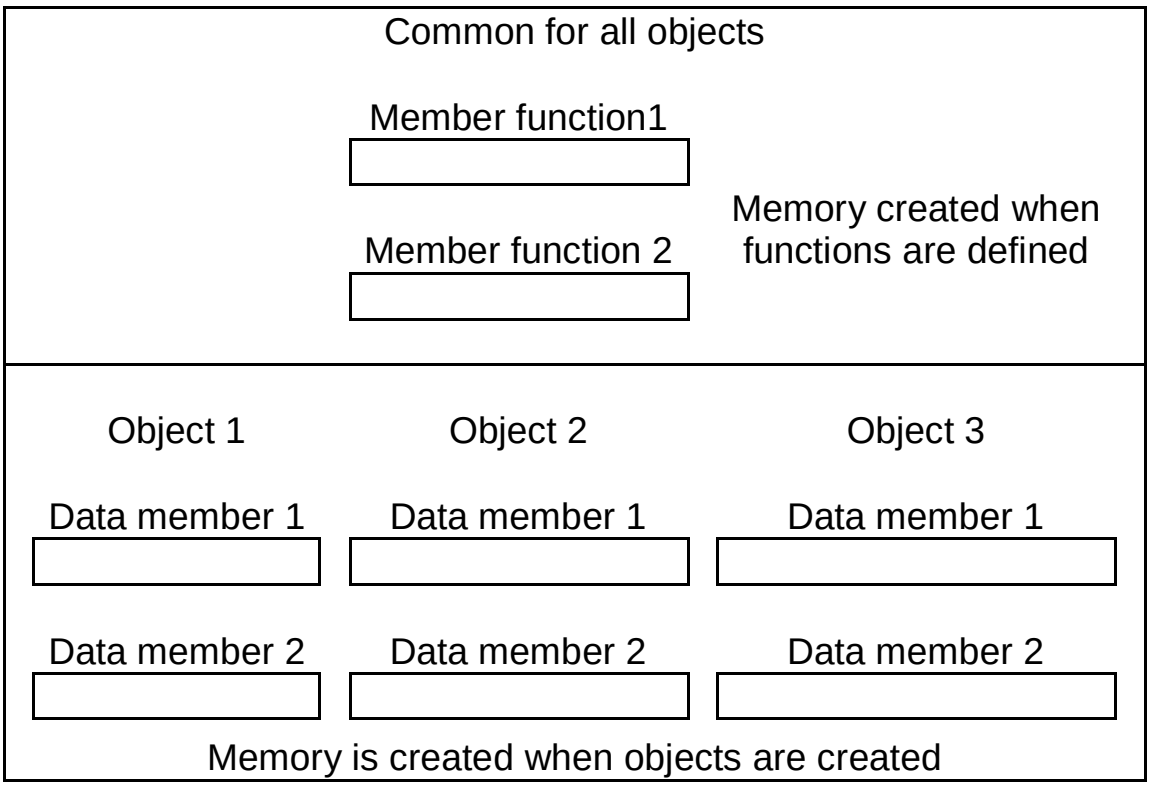
```
#include<iostream>
using namespace std;
class ArrayDemo{
   int a[6];
public:
   void getnos();
   void show();
};
void getnos(){
       cout<<"Enter 6 nos in the array:"<<endl;
       for(int i=0;i<6;i++)
          cin>>a[i];
   }
void show(){
       cout<<"\nArray elements are"<<endl;
       for(int i=0;i<6;i++)
           cout<<a[i]<< "\t";;
    }
```

```
void main(){
   ArrayDemo obj;
   obj.getnos();
   obj.show();
}
```

Output:

```
Enter 6 nos in the array:
5
6
2
1
7
4
Array elements are
5    6    2    1    7    4
```

Memory Allocation for Objects: Member functions are created and placed in the memory space only once when they are defined as a part of a class specification, since all the objects belonging to the class use the same member function. But space for data members is allocated separately for each object at the time of creation of an object. This is shown below.



Static Data Members: Static data members are class members that are declared using the static keyword. A static data member has special characteristics. These are

- Only one copy of a static data member is created for the entire class and is shared by all the objects of that class, no matter how many objects are created.
- The static data member is always initialized to zero when the first class object is created.
- It is visible only within the class, but its lifetime is the entire program.

Syntax: static data_type data_member_name;

Example:

```
#include<iostream>
using namespace std;
class StaticDemo {
      int num;
   public:
      static int count;
      void getdata(int n, int c) {    num = n;    count = c;   }
    void dsiplay() {
      cout<<"num="<< num<<"\t"<<"count="<<count<<endl;
    }
};
int StaticDemo::count;
int main() {
   StaticDemo s1;   s1.getdata(1,1);
   cout<<"Object-1:\t"; s1.dsiplay();
   StaticDemo s2;     s2.getdata(2,2);
   cout<<"Object-2:\t"; s2.dsiplay();
   cout<<"Object-1:\t"; s1.dsiplay();
   return 0;
}
```

Output:

```
Object-1:   num=1   count=1
Object-2:   num=2   count=2
Object-1:   num=1   count=2
```

Static Member Functions: A static member function can be called even if no objects of the class exist. Astatic member function has the following popoties:

- A static member function can access only static data member and other static member functions declared in the same class.
- A static member function can be called using the class name and the scope resolution operator (::) instead of its objects.
- Static member functions have a class scope and they do not have access to the **this** pointer of the class.

Example:

```
#include<iostream>
using namespace std;
class StaticDemo {
        int num;
   public:
      static int count;
      void setNum() {    num = ++count;   }
      void showNum() {
          cout<<"num="<< num <<endl;
      }
     static void showCount() {
      cout<<"Count="<< count <<endl;
    }
};
int StaticDemo::count;
int main() {
   StaticDemo s1, s2;
   s1.setNum();  s2.setNum();
   StaticDemo :: showCount();//static function called
   cout<<"Object-1:\t";   s1.showNum();
   cout<<"Object-2:\t";   s2.showNum();
   return 0;
}
Output:
Count=2
Object-1:    num=1
Object-2:    num=2
```

Arrays of Objects: Like array of other user-defined data types, an array of type class can also be created. The array of type class contains the objects of the class as its individual elements. Thus, an array of a class type is also known as an array of objects. An array of objects is declared in the same way as an array of any built-in data type.

Syntax: class_name array_name [size] ;

Example:

```
#include<iostream>
using namespace std;
class MyClass {
  int x;
public:
  void setX(int i) { x = i+1; }
  int putX() { return x; }
};
int main() {
  MyClass obj[4];
  int i;
  for(i=0; i < 3; i++)
      obj[i].setX(i);
  for(i=0; i < 3; i++)
      cout << "obj[" << i << "]: " << obs[i].putX() << endl;
  return 0;
}
Output:
obj[0]=1
obj[1]=2
obj[2]=3
```

Objects as Function Arguments: Similar to variables, object can be passed to functions. The following are the three methods to pass argument to a function:

a. **Pass-by-value** – A copy of the actual object is sent to function and assigned to the object of callee function (formal object). Both actual and formal copies of objects are stored at different memory locations. Hence, changes made in formal object are not reflected to actual object.

b. **Pass-by-reference** – Address of object is implicitly sent to function
c. **Pass-by-address** – Address of object is explicitly sent of function.

In pass-by-reference and pass-by-address methods, an address of actual object is passed to the function. The formal argument is reference/pointer to the actual object. Hence, changes made in the object are reflected to actual object.

Example - C++ program to add two complex numbers by passing objects to a function.

```
#include<iostream>
using namespace std;
class Complex  {
   private:
     int real, imag;
   public:
    void readData()   {
        cout << "Enter real and imag part:"<<endl;
         cin >> real >> imag;       }
      void addComplexNumbers(Complex c)  {
         real=real+c.real;      imag=imag+c.imag;       }
      void displaySum()  {
         cout << "Sum = "<<real<<"+"<<imag<<"i";     }
};
```

```
int main()  {
   Complex c1,c2,c3;
   c1.readData();    c2.readData();
   c1.addComplexNumbers(c2);
   c1.displaySum();
   return 0;  }
Output
Enter real and imag part:
2
4
Enter real and imag part:
-3
4
Sum = -1+8i
```

Example - C++ program to add two complex numbers using friend function.

```
#include<iostream>
using namespace std;
class Complex {
   private:
     int real, imag;
   public:
    void readData()   {
        cout << "Enter real and imag part:"<<endl;
         cin >> real >> imag;  }
     friend Complex add(Complex c1, Complex c2 )  {
           Complex c3;
           c3.real=c1.real+c2.real;
           c3.imag=c1.imag+c2.imag;
           return c3;  }
      void displaySum()  {
         cout << "Sum = "<<real<<"+"<<imag<<"i";   }
};
```

```
int main() {
   Complex c1,c2,c3;
   c1.readData();
   c2.readData();
   c3=add(c1,c2);
   c3.displaySum();
   return 0;
}
Output
Enter real and imag part:
2
4
Enter real and imag part:
-3
4
Sum = -1+8i
```

Example - C++ program to add two time object using friend function.

```
#include<iostream>
using namespace std;
class Time  {
   private:
     int hour, min;
   public:
    void readData()   {
        cout << "Enter hour and min:"<<endl;
         cin >> hour >> min;  }
     friend Time add(Time t1, Time t2 )  {
           int hr, m;
           Time t3;
           m= t1.min+t2.min
           t3.hour = t1.hour + t2.hour + m/60;
           t3.min=m%60;              return t3;       }
      void displaySum()  {
        cout << "Sum = "<<hour<<" hr "<<min<<" min";   }
};
```

```
int main()  {
    Time t1,t2,t3;
    t1.readData();
    t2.readData();
    t3=add(t1,t2);
    t3.displaySum();
    return 0;
}
```

Output:

```
Enter hour and min:
2
45
Enter hour and min:
3
50
Sum = 6 hour 35 min
```

Constructors & Destructors:

Constructors: A **constructor** is a special member function of a class that is executed whenever we create new objects of that class. Constructors are useful for setting initial values for certain member variables. Constructors have the following properites.

- Constructors have the same name as that of the class to which it belongs.
- **They do not have any return type; not even void.**
- They get automatically invoked when the objects are created.
- If we do not specify a constructor, C++ compiler generates a default constructor for us (expects no parameters and has an empty body).
- They should be declared in the public section.
- They cannot be inherited though derived class can call the base class constructor.
- Like other functions, they can have default arguments
- Constructors can not be virtual.

Example –

```
#include<iostream>
using namespace std;
class Line {
 public:
     Line();  // This is the constructor
};
Line::Line(void) {
   cout<<"Object is being created"<<endl;
}
```

```
int main() {
   Line line;
   return 0;
}
```

Output −

```
Object is being created
```

Types of constructors: C++ offers four types of constructors. These are:

- Do nothing constructor

- Default constructor
- Parameterized constructor
- Copy constructor

Do nothing Constructor: Do nothing constructors are that type of constructor which does not contain any statements. Do nothing constructor is the one which has no argument in it.

Default Constructor: The default constructor is the constructor which doesn't take any argument. It has no parameter but a programmer can write some initialization statement there.

Parameterized Constructor: A default constructor does not have any parameter, but if we need, a constructor to have parameters. This helps us to assign initial value to an object at the time of its creation as shown in the following example –

```
#include<iostream>
using namespace std;
class Line {
      double length;
   public:
     Line(double len);  // This is the constructor
     double getLen( void );
};
Line::Line( double len) {
   cout<<"Object created.";
   length = len;
}
double Line::getLength( void ) {
   return length;
}
int main() {
   Line line(10.0);
   cout<<"Length of line: "<<line.getLen()<<"\n";
   return 0;
}
Output -
Object created.
Length of line: 10
```

Constructors with Default Arguments: It is possible to define constructors with default argumets. For example, the constructor Complex() can be declared as folows:

Complex(float real, float imag=0);

The default value oof the argument imag is zero. Then, the statement

Complex c(5.0);

assigns the value 5.0 to the real variable and 0.0 to imag (by default). However, the statement

complex c(2.0,3.0);

assigns 2.0 to real and 3.0 to imag. The actual parameter, whgen specified, overrides the default value.

Copy Constructor: The copy constructor is a constructor which creates an object by initializing it with an object of the same class, which has been created previously. If a copy constructor is not defined in a class, the compiler itself defines one. But, if the class has pointer variables and has some dynamic memory allocations, then the programmer has to define a copy constructor explicitly.

Example:

```
#include<iostream>
using namespace std;
class Code {
      int id;
   public:
      Code() { }                          // Constructor
      Code(int a) { id = a;  }            // Constructor
      Code(Code &x) { id = x.id;  } // Copy Constructor
      void show() {    cout<<id;    }
};
int main() {
    Code A(100);
    Code B(A); // Copy Constructor called
    Code C=A; // Copy Constructor called
    Code D;  // D is created, not initialized
    D = A; // Copy Constructor NOT called
    cout<<"\n id of  A: "; A.show();
    cout<<"\n id of  B: "; B.show();
    cout<<"\n id of  C: "; C.show();
    cout<<"\n id of  D: "; D.show();
    return 0;
}
```

Output:

```
id of  A: 100
id of  B: 100
id of  C: 100
id of  D: 100
```

Dynamic Initialization of Objects: It means to initialize the data members of the class at run time whereas static initialization occurs at compile time.

Example:

```
#include<iostream>
using namespace std;
class Student {
         int rNo;    float perc;
     public:
         Student() {     }
         Student(int r, float p) {
          rNo = r;
          perc = p;
    }
   void print() {
      cout<<"\nRoll number: "<<rNo;
      cout<<"\tPecentage: "<<perc<<"%";
    }
};
int main() {
    Student s1(1,82.3);        // Static initializaiton
    s1.print();
    cout<<"\nEnter roll number to initialize: ";
    int r;          cin>>r;
    cout<<"\nEnter percentage to initialize: ";
    float p;       cin>>p;
    Student s2;
    s2 = Student(r, p);      // Dynamic initialization
    s2.print();
    return 0;
}
```

Output:

```
Roll number: 1       Pecentage: 82.3%
Enter roll number to initialize: 2
Enter percentage to initialize: 70.5
Roll number: 2       Pecentage: 70.5%
```

Dynamic Constructor: The constructors can also be used to allocate memory while creating objects. This enables the system to allocate the right amount of memory for each object when the objects are not of same size, thus resulting in the saving of memory. Allocation of memory to the objects at the time of their construction is known as dynamic construction of objects. The memory is allocated with the help of **new** operator.

Example-

```
#include<iostream>
#include<string>
using namespace std;
#include<string>
class String {
    char *name;
    int length;
 public :
   String()    {  length = 0;      name = new char[length + 1];    }
   String(char *s)    {
        length = strlen(s);
        name = new char[length + 1]; // One more character for \0
        strcpy(name,s);    }
   void show()    {     cout << name << endl;    }
   void join(String &a, String &b)  {
        length = a.length + b.length;
        delete name;
        name = new char[length + 1]; // dynamic allocation
        strcpy(name, a.name);
        strcat(name, b.name);    }
};
```

```
int main ()  {
  char *first = "Hello ";
  String name1(first);
  String name2("Tia ");

  String s1;
  s1.join(name1, name2);
  name1.show();
  name2.show();
  s1.show();
  return 0;
}
```

Output:

```
Hello
Tia
Hello Tia
```

Destructors: A **destructor** is a special member function of a class that is executed whenever an object of its class goes out of scope or whenever the delete expression is applied to a pointer to the object of that class. Destructor can be very useful for releasing resources before coming out of the program like closing files, releasing memories etc.

Properties of destructor:

- A destructor will have exact same name as the class prefixed with a tilde (~).
- A destructor never takes any argument nor does it return any value.
- Whenever **new** is used to allocate memory in the constructors, we shuld use **delete** to free that memory, because when the pointers to objects go out of scope, a destructor is not called implicitly.
- **Objects are destroyed in the reverse order of creation.**

Following example explains the concept of destructor –

```
#include<iostream>
#include<string>
using namespace std;
int count = 0;
class A {
  public:
    A() { count++;    cout<<"\nNo. of object created "<<count;   }
    //    destructor definition
    ~A(){ cout<<"\nObject No-"<<count<<" deleted"; count--; }
};
int main() {
   A a1, a2;
   {      A a3;    }          {      A a4;    }
  return 0;
}
```

Output −

```
No. of object created 1
No. of object created 2
No. of object created 3
Object No-3 deleted
No. of object created 3
Object No-3 deleted
Object No-2 deleted
Object No-1 deleted
```